

A. Ζητούμενο

Στο εργαστήριο έχετε κατασκευάσει συντακτικό αναλυτή LL(1) με τη μέθοδο αναδρομικής κατάβασης (συν κάποιες τροποποιήσεις, π.χ. για την σωστή προτεραιότητα των τελεστών) για τη γλώσσα των αριθμητικών εκφράσεων που περιγράφεται από την εξής γραμματική:

```
Stmt_list → Stmt Stmt_list | ε
Stmt → id = Lexpr | print Lexpr
      | if Lexpr Block_stmt (else Block_stmt)?
      | while Lexpr Block_stmt
Block_stmt → Stmt | { Stmt_list }
Lexpr → Lterm Lterm_tail
Lterm_tail → or Lterm Lterm_tail | ε
Lterm → Lfactor Lfactor_tail
Lfactor_tail → and Lfactor Lfactor_tail | ε
Lfactor → not Lfactor | Expr Rest
Rest → Relop Expr | ε
Expr → Term (Addop Term)*
Term → Factor (Multop Factor)*
Factor → (Lexpr) | id | number
Addop → + | -
Multop → * | /
Relop → == | != | > | >= | < | <=
```

Μια ενδεικτική υλοποίηση του συντακτικού αναλυτή, μαζί με έναν διερμηνευτή του δένδρου AST που προκύπτει μπορείτε να βρείτε στο:

<https://mixstef.github.io/courses/compilers/if-while.py>

Ως βάση της εργασίας θα πρέπει να χρησιμοποιήσετε είτε το πιο πάνω υπόδειγμα, είτε τον αντίστοιχο κώδικα που εσείς αναπτύξατε στο εργαστήριο. Η τροποποίηση του υπάρχοντα κώδικα στον ελάχιστο δυνατό βαθμό θα θεωρηθεί κριτήριο επιτυχίας στην επίλυση της εργασίας.

Το ζητούμενο της εργασίας είναι η προσθήκη του τελεστή *ανάθεσης* σε μεταβλητή μέσα σε μια *έκφραση*, ο οποίος θα συμβολίζεται από το token `:=`

Η λειτουργικότητα του τελεστή αυτού είναι παρόμοια με του τελεστή “walrus” της Python (βλ. π.χ. <https://docs.python.org/3/whatsnew/3.8.html#assignment-expressions>).

Οι κανόνες σύνταξης του νέου τελεστή θα είναι οι εξής:

1. Ο τελεστής `:=` μπορεί να εμφανιστεί **μόνο μία φορά** μέσα σε μια έκφραση (Lexpr στη γραμματική) και **πάντα στην κεφαλή της έκφρασης**.
Π.χ. το: `print a := 8*b+4` είναι έγκυρο, ενώ το `print 8 * a:= b+4` ή το `print a:= 8 * c:= b+4` αποτελούν άκυρη σύνταξη.
2. Εξαίρεση στο (1) αποτελεί η επανεμφάνιση του `:=` σε **υποέκφραση μέσα σε παρενθέσεις**, που είναι έγκυρη. Π.χ. το `print a:= 8 * (c:= b+4)` είναι απολύτως έγκυρο ως σύνταξη.

B. Διαδικασία επίλυσης

Ακολουθήστε τα παρακάτω βήματα για τη δημιουργία του κώδικα υλοποίησης σε Python. Οι ερωτήσεις (με αριθμηση E.x) που τίθενται σε κάθε στάδιο θα πρέπει να απαντηθούν στη συνοδευτική αναφορά που θα παραδώσετε μαζί με τον κώδικα.

1. Προσθήκη token του νέου τελεστή

(E.1) Σε ποια θέση στη σειρά δήλωσης των διαφόρων tokens στον λεκτικό αναλυτή θα προσθέσετε τον νέο τελεστή; Έχει σημασία η σειρά δήλωσης του τελεστή αυτού σε σχέση με τα υπάρχοντα tokens;

2. Τροποποίηση γραμματικής

Στη συνέχεια θα πρέπει να διερευνήσετε με ποιον τρόπο θα τροποποιηθεί η νέα γραμματική για να συμπεριλάβει τον νέο τελεστή.

(E.2) Ως πρώτο βήμα για τον εντοπισμό του σημείου προσθήκης κανόνων για τον νέο τελεστή, θα πρέπει να βρείτε την προτεραιότητα του τελεστή σε σχέση με τους υπάρχοντες τελεστές. Σε ποιο επίπεδο προτεραιότητας βρίσκεται; Αναφερθείτε στη λειτουργία του τελεστή (με παραδείγματα έκφρασης που τον περιέχει) για να απαντήσετε.

(E.3) Ποιοι είναι οι νέοι κανόνες που πρέπει να προστεθούν;

(E.4) Μετά την προσθήκη των νέων κανόνων προκύπτει γραμματική LL(1); Τεκμηριώστε αναλυτικά την απάντησή σας, είτε είναι ναι είτε όχι. Για τον έλεγχο των FIRST/FOLLOW sets μπορείτε να χρησιμοποιήσετε ένα εργαλείο όπως π.χ. εδώ: <https://smlweb.cpsc.ucalgary.ca/start.html>.

(E.5) Αν η απάντηση στο (E.4) είναι όχι, περιγράψτε πώς σκοπεύετε να το αντιμετωπίσετε, περιγράφοντας έναν πιθανό μηχανισμό παράκαμψης του προβλήματος, ο οποίος όμως θα πρέπει να επιφέρει όσο το δυνατόν λιγότερες αλλαγές στον υπάρχοντα κώδικα.

3. Τροποποίηση κώδικα συντακτικού αναλυτή

(E.6) Με βάση τις απαντήσεις σας στα (E.2-5) περιγράψτε τις αλλαγές στον κώδικα που θα πραγματοποιήσετε (τι θα προστεθεί, αφαιρεθεί ή θα αλλάξει).

4. Δένδρο AST και διερμηνεία

(E.7) Απαιτείται καινούργιο είδος κόμβου για να καλυφθεί η λειτουργικότητα του νέου τελεστή; Αν ναι, περιγράψτε τα χαρακτηριστικά του (type, attributes, subnodes).

(E.8) Περιγράψτε πώς εκτελείται η λειτουργικότητα του νέου τελεστή στον interpreter του δένδρου AST.

5. Κατασκευή κώδικα και δοκιμαστικού κειμένου εισόδου

Κατασκευάστε τον κώδικα του νέου συντακτικού αναλυτή σύμφωνα με τις απαντήσεις σας στα προηγούμενα ερωτήματα (E.1-8). Κατασκευάστε επίσης δοκιμαστικό κείμενο, το οποίο θα πρέπει να καλύπτει όλες τις πιθανότητες χρήσης του νέου τελεστή.

6. Αναφορές

(E.9) Παραθέστε τις πηγές που χρησιμοποιήσατε για την επίλυση της εργασίας (εκτός από το υλικό του εργαστηρίου).

(E.10) Σε περίπτωση χρήσης εργαλείων TN περιγράψτε σε ποιο στάδιο χρησιμοποιήθηκαν και με ποιον τρόπο.

Γ. Παραδοτέο

i) Όταν έχετε ολοκληρώσει την επίλυση της εργασίας, περάστε τον τελικό κώδικα σε ένα jupyter notebook (αν δεν χρησιμοποιείτε notebook από την αρχή). Εκτελέστε τον συνολικό κώδικα με το κατάλληλο δοκιμαστικό κείμενο εισόδου. Τα αποτελέσματα των δοκιμών θα πρέπει να εκτυπωθούν και να φαίνονται στο τελικό jupyter notebook που θα παραδώσετε.

ii) Ετοιμάστε αναφορά σε μορφή pdf με τις απαντήσεις σας στα ερωτήματα E.x των προηγούμενων ενοτήτων.

Ανεβάστε το τελικό σας notebook και την αναφορά pdf σε ένα μοναδικό αρχείο zip στο opencourses (**Εργασία 2**) έως και τη **Παρασκευή 5/6**.

Βαθμολόγηση εργασίας

Κριτήριο	% βαθμού
Επιτυχής συγγραφή προγράμματος Python με όσο το δυνατόν λιγότερες αλλαγές του υπάρχοντος κώδικα και με ορθή εμφάνιση αποτελεσμάτων	40%
Πλήρεις και τεκμηριωμένες απαντήσεις των ερωτημάτων στην αναφορά	60%